File Permissions in Linux

Portfolio Activity — Manage Authorization

Project Description

As a security professional supporting a large organization's research team, this project involved auditing and correcting Linux file permissions on the **projects** directory. Using **ls -la** to reveal all permissions (including hidden files), I identified misconfigurations and applied **chmod** commands to remove unauthorized write access, enforcing the principle of least privilege across files and directories.

Step 3 — Check File and Directory Details

The **ls -la** command was run inside **/home/researcher2/projects** to list all files and subdirectories including hidden ones. The **-l** flag gives long format output; **-a** includes hidden files (beginning with a dot).

Command:

```
researcher2@linux:~/projects$ ls -la
```

Output:

```
total 32
drwxr-xr-x 3 researcher2 research_team 4096 May 01 09:00 .
drwxr-xr-x 8 researcher2 research_team 4096 May 01 09:00 ..
drwx--x--- 2 researcher2 research_team 4096 May 01 09:00 drafts
-rw-rw-rw- 1 researcher2 research_team 46 May 01 09:00 project_k.txt
-rw-r--r-- 1 researcher2 research_team 46 May 01 09:00 project_m.txt
-rw-rw-rw- 1 researcher2 research_team 46 May 01 09:00 project_r.txt
-rw--w--w- 1 researcher2 research_team 46 May 01 09:00 .project_x.txt
```

Step 4 — Describe the Permissions String

Example: **-rw-rw-rw-** (project_k.txt)

Position(s)	Char(s)	Meaning
1	-	Regular file (not a directory)
2–4	rw-	User (owner): read + write; no execute
5–7	rw-	Group: read + write; no execute
8–10	rw-	**Others: read + write — POLICY VIOLATION**

The 'others' write bit (position 9) violates the policy that others must not have write access to any file.

Step 5 — Change File Permissions

project_k.txt and **project_r.txt** both had **-rw-rw-rw-**. The **chmod o-w** command removes write access from 'others'.

Commands:

```
researcher2@linux:~/projects$ chmod o-w project_k.txt
researcher2@linux:~/projects$ chmod o-w project_r.txt
researcher2@linux:~/projects$ ls -la
```

Result:

```
-rw-rw-r-- 1 researcher2 research_team 46 May 01 09:02 project_k.txt
-rw-rw-r-- 1 researcher2 research_team 46 May 01 09:02 project_r.txt
```

Others now have read-only access. Policy satisfied.

Step 6 — Change Permissions on a Hidden File

.project_x.txt had **-rw--w--w-**. Requirement: no write permissions for anyone; user and group may read. Target: **-r--r-----**.

Command:

```
researcher2@linux:~/projects$ chmod u-w,g-w,g+r,o-w,o-r .project_x.txt
researcher2@linux:~/projects$ ls -la .project_x.txt
```

Result:

```
-r--r----- 1 researcher2 research_team 46 May 01 09:05 .project_x.txt
```

Hidden archived file now allows read-only for owner and group; no write permissions exist for any category.

Step 7 — Change Directory Permissions

drafts/ had **drwx--x---**. Only researcher2 (the user) should have execute access. The group's execute bit must be removed.

Command:

```
researcher2@linux:~/projects$ chmod g-x drafts
researcher2@linux:~/projects$ ls -la
```

Result:

```
drwx------ 2 researcher2 research_team 4096 May 01 09:08 drafts
```

The directory is restricted to the owner only; the group can no longer navigate into drafts.

Summary

Using **ls -la** to audit the **projects** directory, I identified four misconfigurations: two regular files allowed others to write, one hidden archived file had write permissions for all parties, and one directory allowed the group execute access. Targeted **chmod** commands corrected each issue, verified by follow-up **ls -la** output. This demonstrates how Linux permission management enforces least privilege and protects sensitive research data.